Agentized Graph Reasoning via Policy-Guided Deduction

Paper ID: YYY

ABSTRACT

This paper studies agentized graph reasoning, where deduction is conducted as a goal-directed sequence of controlled rule applications rather than exhaustive closure. The key challenge is selecting the next inference step. We introduce *policy-guided deduction*, in which a policy maps the current state and goal to a reasoning motif, to efficiently advance toward the target goal. We formalize the *policy selection problem* over Graph Association Rules, define a policy space, and propose a unified framework. This formulation covers both optimal reasoning (minimizing proof cost) and anytime reasoning (maximizing utility under constraints). We show that policy selection is intractable. Nonetheless, we develop effective policy strategies: deterministic, randomized, learned, and hybrid neuro-symbolic, guaranteeing soundness, goal-level completeness and approximation bounds. Using real-life datasets and workloads, we experimentally verify that policy-guided reasoning effectively reduces proof size and runtime, achieves near-optimal utility under tight budgets, and generalizes across diverse tasks.

1 INTRODUCTION

Rule-based graph reasoning is widely used in data-intensive applications such as anti-money laundering (AML), fraud detection, cybersecurity and risk analytics. In these domains, domain knowledge is encoded as rules (*e.g.,* Graph Association Rules, GARs [17]) that match graph patterns and derive new facts [18, 19], explanations [15, 41] or risks [44], by integrating logical reasoning with machine learning (ML). Given a graph and a rule set, classical reasoning systems apply rules to infer conclusions, typically via exhaustive mechanisms such as chase [2, 9, 17], which compute the full logical closure.

Example 1: Consider an AML task over a financial transaction graph G. Vertices represent customers and accounts, while edges denote ownership and fund transfers. Suppose that an analyst asks: *Why is customer u flagged as high risk?* A typical explanation requires a reasoning chain such as: (a) u owns account a_1; (b) a_1 transfers a large amount to account a_2; (c) a_2 is linked to an identified high-risk entity; (d) risk propagates back to u. This chain provides a concise and understandable justification for the decision.

However, under rule-based reasoning, the system would not construct such a focused explanation. Instead, it would exhaustively apply all applicable rules on the graph, deriving many intermediate facts irrelevant to the query: unrelated transfer paths, expanded account clusters, risk signals propagated across distant parts of G, incurring substantial reasoning cost and opaque outcome. □

This example motivates a paradigm shift in modern graph analytics. Reasoning is no longer by enumerating all logical consequences, but by constructing *goal-directed, decision-relevant derivations*. Analysts require concise, interpretable reasoning chains that justify target facts, often under strict latency or resource constraints. In practice analysts want the most useful evidence within a budget; a minimum-cost chain is the counterpart of that utility for explanation quality.

The observations suggest that the key challenge in modern reasoning is not only *what* can be derived, but also *how* to derive it efficiently and purposefully. We need a mechanism to control the reasoning process, deciding which inference step to perform next in order to best support the target goal. That is, we model reasoning as a controlled, goal-directed and *agent-driven* process.

Limitations of prior approaches. Forward-closure systems, the chase and its engines [2, 6, 7, 9, 17, 21, 30], fall short in following. .

Not goal-directed. Forward closure derives all logical consequences, whether or not they serve a target query: in the AML example, excessive irrelevant transfer paths, misaligned with explaining why a specific customer is flagged.

Not cost-aware. Forward closure has no notion of cost: it neither controls proof size nor optimizes latency, and recomputes redundant derivations, prohibitively costly in large graphs.

No control abstraction. Closure applies rules whenever they apply; there is no decision layer that chooses *which rule to apply next*, one that a goal or a budget could optimize.

Goal-directed evaluation answers what is derivable toward g without the full closure, via magic-set rewriting [8], query/subquery [54] or tabling [48]; but it prices nothing and controls no budget. Learned reasoners such as NBFNet [59] and A*Net [58] score answers toward a query, but produce no certified logical derivations. What is missing is budgeted, cost-certified, goal-directed deduction.

Policy-based agentized deduction. The limitations suggest that the bottleneck is not expressiveness, but control: reasoning lacks a systematic, adaptive mechanism to guide inference. This motivates viewing reasoning as a *decision process*. We model reasoning as a sequence of decisions, each selecting the next inference step based on the current state and goal. Formally, we introduce a *policy*

$$\pi : (S, g) \rightarrow \mu,$$

which maps a deduction state S and a target goal g to a reasoning motif μ (*i.e.,* a rule application instance), and induces a *reasoning chain* from an initial state S_0 toward g. Beyond a fixed policy, modern reasoning requires *adaptive* control: the next step depends on the state, goal and resource constraints, calling for an *agent* that dynamically selects and adapts policies rather than following a static strategy.

This formulation offers several key advantages.

Decoupling semantics and control. It separates *what* can be derived (defined by GAR semantics and chase) from *how* it is derived (controlled by π), preserving logical correctness while enabling goal-directed, cost-aware, and adaptive reasoning.

Unifying optimal and anytime reasoning. An *optimal policy* minimizes proof cost, yielding concise reasoning chains; and an *anytime policy* maximizes decision utility under resource constraints. Both minimal explanations and budgeted inference are captured within a unified framework.

Agentized deduction. The agent adapts its behavior based on the

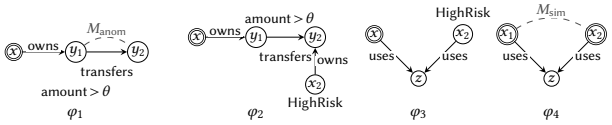


Figure 1: Patterns of the GARs of Example 2.

current state, varying goals, workloads and resource constraints, enabling controllable and explainable reasoning at scale.

Problem statement. Based on this, we study *policy selection problem* for rule-based graph systems. Given a graph G, a set Σ of GARs, initial facts Γ_0, and a goal g, the task is to construct a reasoning chain that derives g. Specifically, the problem is to find a policy $\pi : (S, g) \rightarrow \mu$ that induces a chain of rule applications toward g, starting from the initial state $S_0 = (G, \Sigma, \Gamma_0)$, to optimize two objectives: (a) *optimal reasoning*, and (b) *anytime reasoning*.

Challenges. The problem poses several challenges.

Combinatorial search space. At each state, many rule instantiations are possible, yielding an exponential space of reasoning chains; policy selection is intractable, as we will show.

Non-local dependencies. The utility of a rule depends on downward derivations, making locally optimal choices globally suboptimal.

Soundness and completeness. Each step of the deduction must be sound under GAR semantics, *i.e.,* its preconditions hold and its consequence is entailed; and any entailed goal should remain reachable under suitable policies.

Multi-objective tradeoffs. Policies must balance cost, utility and resource constraints, often under predefined tight budgets.

Contributions & Organization. We study policy selections for policy-guided reasoning, and make the following contributions.

(1) Foundations of agentized reasoning (Section 2). We revisit GAR-based reasoning and formalize *reasoning chains* and *motifs* as the building blocks of goal-directed inference, decoupling *semantics* (GARs and chase) from *control*.

(2) Policy selection as a unifying framework (Section 3). We formulate reasoning as a *policy selection problem* $\pi : (S, g) \rightarrow \mu$, taking the current fact set and the goal as the state, and unify *optimal reasoning* (cost minimization) and *anytime reasoning* (utility maximization under budget). We show that policy selection is NP-complete.

(3) Approximate policy-guided reasoning (Sections 4 and 5). We develop practical policy classes such as deterministic, randomized, learned and hybrid neuro-symbolic policies, together with execution and adaptation mechanisms. These provide a unified framework for *adaptive, budget-aware and workload-aware reasoning.*

(4) Theoretical guarantees (Section 6). We establish a theoretical characterization of policy-guided deduction, including (a) *soundness* and *goal-level completeness w.r.t.* chase, (b) *performance guarantees* for both optimal and anytime reasoning chains (approximation bounds), and (c) *termination, complexity and convergence* results.

(5) Experimental evaluation (Section 7). On real-world datasets and workloads, we show that policy-guided reasoning reduces proof size and latency, achieves near-optimal utility under tight budgets, and generalizes across diverse tasks. The study reports success rate, utility, cost and runtime.

Related and future work are in Sections 8 and 9; proofs are in [1].

2 AGENTIZED GRAPH REASONING

This section reviews the foundations of rule-based graph reasoning. We consider GARs (Section 2.1), and chase as the semantic basis for deriving facts (Section 2.2). We then present reasoning chains for agentized goal-directed reasoning (Section 2.2). These notions characterize what can be derived under GAR semantics and policy-based control of inference (Section 2.3).

2.1 Graph Association Rules

We start with the primitives of graph reasoning.

Graphs and patterns. A *graph* is a labeled directed graph $G = (V, E, L, \mathcal{A})$, where V is a finite set of vertices, $E \subseteq V \times \Lambda_E \times V$ is a finite set of labeled edges, $L : V \rightarrow \Lambda_V$ assigns a label to each vertex, and $\mathcal{A}$ assigns to each vertex a finite tuple of attributes, without a fixed schema. A *pattern* is a graph $Q[\bar{x}] = (V_Q, E_Q, L_Q)$ whose nodes are the variables of a list $\bar{x}$, with labeled edges $E_Q \subseteq V_Q \times \Lambda_E \times V_Q$ and labels of Λ_V on nodes. A *match* of Q in G is a homomorphism $h : V_Q \rightarrow V$ that preserves labels and edges, *i.e.,* $L_Q(x) = L(h(x))$ for all $x \in V_Q$ and $(h(x), l, h(y)) \in E$ for every $(x, l, y) \in E_Q$.

Graph Association Rules. We start with predicates of GARs.

Predicates. A predicate over $Q[\bar{x}]$ is a Boolean condition on the variables $\bar{x}$: (a) $x.A \oplus c$, where $x.A$ denotes the A attribute of $x \in \bar{x}$, $\oplus \in \{=, \neq, <, >, \leq, \geq\}$ and c is a constant; (b) $x.A \oplus y.B$, comparing attributes of two variables; (c) $l(x, y)$, an edge with label l between $h(x)$ and $h(y)$; and (d) $M(x.\bar{A}, y.\bar{B})$, an ML predicate over attribute vectors $\bar{A}$ and $\bar{B}$ of x and y, such as a risk score exceeding a threshold. We call (a) and (b) *attribute predicates*, (c) an *edge predicate* and (d) an *ML predicate.*

GARs. A Graph Association Rule (GAR) is a rule of the form

$$\varphi = Q[\bar{x}](X(\bar{x}) \rightarrow p_0(\bar{x}_0)),$$

where $Q[\bar{x}]$ is a pattern, $X(\bar{x})$ is a finite set of predicates over $Q[\bar{x}]$, called the *preconditions* of φ, and $p_0(\bar{x}_0)$ is a predicate over $Q[\bar{x}]$, called the *consequence*, defined with a subset $\bar{x}_0 \subseteq \bar{x}$ of variables.

Given a match h of Q in G, the predicates of X are evaluated over the matched vertices: attribute predicates compare the attribute values of $h(x)$ and $h(y)$, and an ML predicate $M(x.\bar{A}, y.\bar{B})$ evaluates the pre-trained model M on their attribute vectors. An ML predicate is a deterministic Boolean oracle over the attribute vectors, its answers fixed for the run: entailment (Theorem 2) is relative to those fixed answers, and confidences enter only as features for learned policies downstream.

If all predicates in X hold, the rule derives the consequent fact $p_0(h(\bar{x}_0))$.

Example 2: The reasoning patterns behind the AML task of Example 1 are captured by GARs φ_1–φ_4 below. For brevity, we write $x.A = \text{true}$ as $A(x)$ when clear from context.

(1) Layered transfer risk propagation. Let $Q_1[x, y_1, y_2]$ contain edges $\text{owns}(x, y_1)$ and $\text{transfers}(y_1, y_2)$. Consider GAR

$\varphi_1 : Q_1[x, y_1, y_2](y_1.\text{amount} > \theta \wedge M_{\text{anom}}(y_1, y_2) \rightarrow \text{HighRisk}(x))$.

The rule states that a customer x is high-risk if its account y_1 makes

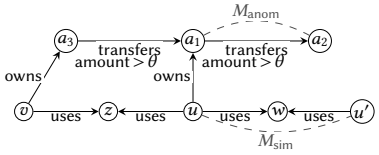


Figure 2: The graph G of Example 3; dashed curves are ML predicates.

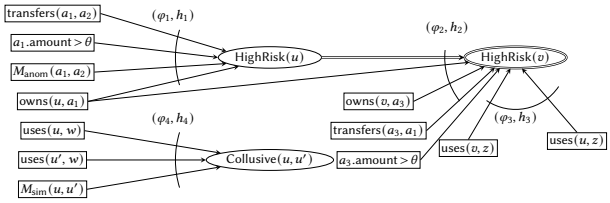


Figure 3: The chains of Example 4 toward $g = \text{HighRisk}(v)$, drawn with shared facts merged: rectangles are facts of Γ_0, ellipses derived facts, the double ellipse the goal. An edge into a fact marks a premise of its derivation; an arc (φ_i, h_i) groups the premises of one application, all of which must hold. The double edge is shared by (φ_2, h_2) and (φ_3, h_3), and the (φ_4, h_4) branch is a dead end, no path to g.

a large anomalous transfer to y_2, where M_{anom} is a pretrained ML model for suspicious behavior.

(2) Risk propagation. Let $Q_2[x, y_1, y_2, x_2]$ contain edges $\text{owns}(x, y_1)$, $\text{transfers}(y_1, y_2)$ and $\text{owns}(x_2, y_2)$. GAR φ_2 propagates risk along transfers: a customer x is high-risk if its account makes a large transfer to an account of a high-risk customer.

$$\varphi_2 : Q_2[x, y_1, y_2, x_2]\big(y_1.\text{amount} > \theta \wedge \text{HighRisk}(x_2) \rightarrow \text{HighRisk}(x)\big).$$

Repeatedly applied, φ_2 propagates risk along transfer chains.

(3) Infrastructure sharing. Let $Q_3[x, x_2, z]$ contain edges $\text{uses}(x, z)$ and $\text{uses}(x_2, z)$. GAR φ_3 propagates risk through shared infrastructure: a customer x is high-risk if it shares infrastructure z with a high-risk customer x_2.

$$\varphi_3 : Q_3[x, x_2, z]\big(\text{HighRisk}(x_2) \rightarrow \text{HighRisk}(x)\big).$$

(4) Cross-entity correlation. Let $Q_4[x_1, x_2, z]$ contain edges $\text{uses}(x_1, z)$ and $\text{uses}(x_2, z)$. GAR φ_4 catches collusive entities that share infrastructure z and exhibit similar behavior under an ML predicate M_{sim}.

$$\varphi_4 : Q_4[x_1, x_2, z]\big(x_1 \neq x_2 \wedge M_{\text{sim}}(x_1, x_2) \rightarrow \text{Collusive}(x_1, x_2)\big),$$

Their patterns are depicted in Figure 1: double circles mark the vertices on which a rule derives its consequent, and dashed curves are ML predicates. □

2.2 Chase-based Deduction

Chase. Let Γ_0 be a finite set of *validated facts* over G, *e.g.*, $\text{owns}(u, a)$, $a.\text{amount} > \theta$ or $M(a_1, a_2) = \text{true}$, collected from data sources. A rule φ is *applicable* on a fact set Γ under a match h of its pattern if all its preconditions X hold under h *w.r.t.* Γ and the consequence $p_0(h(\bar{x}_0))$ is not yet in Γ; applying it derives the fact,

$$\Gamma \Rightarrow_{\varphi,h} \Gamma \cup \{p_0(h(\bar{x}_0))\}.$$

The *chase* repeatedly applies rules of Σ, $\Gamma_0 \Rightarrow \Gamma_1 \Rightarrow \cdots$, until a fixpoint, and returns the *closure* $\Gamma^* = \text{Chase}(G, \Sigma, \Gamma_0)$, all facts entailed by Σ over G. It terminates and is *confluent* (Church-Rosser, cf. [2]), so Γ^* is unique regardless of the order of applications; and the derived facts are logical consequences of Γ_0 and Σ, hence correct when those are [17].

Example 3: Continuing Example 2, consider a graph G with customers u, u', v, accounts a_1, a_2, a_3 and shared facilities z, w (Figure 2). Let Γ_0 consist of twelve facts: $\text{owns}(u, a_1)$, $\text{transfers}(a_1, a_2)$, $a_1.\text{amount} > \theta$, $M_{\text{anom}}(a_1, a_2)$, $\text{owns}(v, a_3)$, $\text{transfers}(a_3, a_1)$, $a_3.\text{amount} > \theta$, $\text{uses}(v, z)$, $\text{uses}(u, z)$, $\text{uses}(u, w)$, $\text{uses}(u', w)$ and $M_{\text{sim}}(u, u')$. The chase derives $\text{HighRisk}(u)$ via φ_1 under $h_1 = \{x \mapsto u, y_1 \mapsto a_1, y_2 \mapsto a_2\}$. It then derives $\text{HighRisk}(v)$ via φ_2 under $h_2 = \{x \mapsto v, y_1 \mapsto a_3, y_2 \mapsto a_1, x_2 \mapsto u\}$ and via φ_3 under $h_3 = \{x \mapsto v, x_2 \mapsto u, z \mapsto z\}$, both resting on the fact $\text{HighRisk}(u)$ just derived; and, independently of any goal, $\text{Collusive}(u, u')$ via φ_4. Nothing more is derivable: the closure is $\Gamma^* = \Gamma_0 \cup \{\text{HighRisk}(u), \text{HighRisk}(v), \text{Collusive}(u, u')\}$ in any order of the applications, and $\text{Collusive}(u, u')$ can never support $\text{HighRisk}(v)$. □

Reasoning chains. Given a goal g and initial facts Γ_0, a *reasoning chain* is a sequence that applies only rule instances relevant to g:

$$C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle,$$

where each $\varphi_i \in \Sigma$ is applied under a match h_i to extend the current fact set, and the final step derives g, as opposed to chase.

Example 4: Continuing Examples 2 and 3, let $\Sigma = \{\varphi_1, \varphi_2, \varphi_3, \varphi_4\}$, Γ_0 be the twelve facts of Example 3, and $g = \text{HighRisk}(v)$. The chains toward g, drawn together in Figure 3, derive three facts on top of Γ_0: (φ_1, h_1) derives $\text{HighRisk}(u)$ from the four facts its arc crosses; and building on it, (φ_2, h_2) and (φ_3, h_3) derive the goal $\text{HighRisk}(v)$ in two ways, their arcs sharing the edge out of $\text{HighRisk}(u)$, which only (φ_1, h_1) derives. Off the goal, (φ_4, h_4) derives $\text{Collusive}(u, u')$, a dead end: no rule takes Collusive as a premise, so no path from it reaches g. The fifteen facts at play are exactly the closure Γ^*. □

Reasoning chains give concise, goal-directed explanations.

2.3 From Semantics to Control

We have defined *what* can be derived under GAR semantics via chase and reasoning chains. We now turn to *how* to derive facts.

Semantics vs. control. Chase provides a complete and confluent characterization of all entailed facts, but it is exhaustive, task-agnostic and expensive. Reasoning chains restrict derivations to a goal, but they do not specify how to choose the next rule application. Thus, existing models capture semantics but lack a *control* mechanism.

Control. We model control via an *agent* that interacts with the rule system: given a state $S = (G, \Sigma, \Gamma)$ and a goal g, a *policy* $\pi : (S, g) \rightarrow \mu$ picks the next rule application, and starting from $S_0 = (G, \Sigma, \Gamma_0)$ it induces a reasoning chain. All such chains, bounded by the closure $\text{Chase}(G, \Sigma, \Gamma_0)$, form the space that a policy searches. This formulation *decouples* semantics from control: GARs and chase define what is entailed, while the agent, via π, determines how inference proceeds, so policies can be goal-directed, cost-aware and adaptive without altering logical semantics.

3 THE POLICY SELECTION PROBLEM

This section formalizes policy-guided reasoning as an *agent-based decision model* over rule systems. We define the agent model (Section 3.1), and formulate policy selection as an optimization problem with cost and utility objectives (Section 3.2). We prove its intractability (Section 3.3).

3.1 Agent Model

We model policy-guided reasoning as an *agent* interacting with the rule system.

Actions (motifs). We model actions as *motifs*, *candidate* rule applications whose preconditions are yet untested. Formally, a motif μ is a pair (φ, h), where $\varphi \in \Sigma$ is a GAR and h is a match of its pattern in G, by labels and edges alone. Applying μ tests the preconditions of φ under h: if all hold, the application *succeeds* and the state extends to $S' = (G, \Sigma, \Gamma \cup \{p_0(h(\bar{x}_0))\})$; if some fail, nothing is derived, and the failure is recorded. Write $\mathcal{A}(S)$ for the motifs available at S, keeping out those whose consequent is already in Γ, deriving nothing new, and those excluded by a recorded failure, as follows.

Agent dynamics. Starting from the initial state $S_0 = (G, \Sigma, \Gamma_0)$, the agent iteratively applies motifs, inducing a reasoning chain toward g. How a failure is recorded in $\mathcal{A}(S)$ depends on whether later derivations can change the failed precondition. A precondition is *static* when no rule of Σ derives its predicate, *e.g.,* amount $> \theta$, an edge of a non-derived relation, or an ML predicate, whose oracle answers are fixed for the run (Section 2.1): such tests read G only, and G never changes, so a motif failing on one fails at every later state and is retired from $\mathcal{A}(S)$ for good. It is *dynamic* when some rule derives its predicate, *e.g.,* HighRisk(u) for (φ_2, h_2) or an edge of a derived relation: a later application can make it hold, and since facts are only added, it flips only from false to true. The failure is then recorded against the current fact set, and the motif is re-admitted to $\mathcal{A}(S)$ once the failed predicate gains a fact in Γ. Retiring static failures loses nothing, and re-admitting dynamic ones keeps fair policies complete (Theorem 3), while trying each motif at most once per fact set keeps the search finite (Theorem 7).

3.2 Problem Statement

We next formalize policy selection, starting with objectives.

Objective functions. Given an initial state $S_0 = (G, \Sigma, \Gamma_0)$ and a goal g, a policy π induces a reasoning chain $S_0 \xrightarrow{\mu_1} S_1 \xrightarrow{\mu_2} \cdots \xrightarrow{\mu_k} S_k$, denoted $C(\pi; S_0, g)$. We evaluate π by a cost function over the chain and a utility function toward g under resource constraints.

Cost minimization (optimal reasoning). We want to have a policy that derives g with minimum cost. Every application of a motif, successful or failed, pays for the evaluation of its preconditions. Write $\text{cost}(\mu, S)$ for the cost of applying μ at state S: $\text{cost}(\mu, S) \geq w_0(\varphi) \geq 1$, where $w_0(\varphi)$ is a certified lower bound on the cost of one application of φ; under unit cost, each application costs exactly 1. The total cost of a chain is

$$\text{cost}(C) = \sum_{i=1}^{k} \text{cost}(\mu_i, S_{i-1}).$$

The budget B is per chain: each branch accumulates the cost of its own applications, successful or failed, and is feasible iff the total is at most B. Candidate generation, scoring and decision-model calls are control overhead: wall-clock, reported separately in the experiments (Section 7), not charged to B. Speculative work is bounded by termination; B certifies the chain returned. For randomized π, we minimize expected cost:

$$\min_{\pi} \ \mathbb{E}_{C\sim\pi}\big[\text{cost}(C)\big] \quad \text{such that } g \in \Gamma_k.$$

Utility maximization (anytime reasoning). The budget B may be too small to derive g. Still, the agent should make progress *anytime*: whenever it stops, the facts derived so far should be worth as much as possible. Let $\Gamma_{\leq B}$ be the fact set π derives before the budget B runs out. We measure progress toward g by *subgoals*, *i.e.,* facts that can appear on some reasoning chain deriving g, g included; write $\mathcal{S}_g$ for their set, and call a subgoal *realized* in Γ when the fact is in Γ. The utility is a *weighted coverage* of subgoals,

$$U(\Gamma, g) = \textstyle\sum_{q\in\mathcal{S}_g,\, q \text{ realized in } \Gamma} w_q,$$

where $w_q \geq 0$ is the weight of subgoal q, decreasing with the length of the shortest chain from q to g. It is *monotone and submodular*: more facts never hurt, and a subgoal counts only when first derived, giving greedy search a guarantee (Theorem 6). With all weight on g, it is just *goal satisfaction*. We solve

$$\max_{\pi} \ \mathbb{E}\big[U(\Gamma_{\leq B}, g)\big] \quad \text{such that } \text{cost}(C) \leq B.$$

Multi-objective optimization The two objectives trade off against each other. A common combination earns utility but pays λ per unit of cost,

$$\max_{\pi} \ \mathbb{E}\big[U(\Gamma_{\leq B}, g) - \lambda\, \text{cost}(C)\big],$$

for a parameter $\lambda \geq 0$.

This objective scores whole chains, but the agent decides step by step, so we read it per step. Write S_μ for the state that applying μ at S leads to, and Γ_μ for its fact set. The step earns its *marginal utility* $\Delta U(S, g, \mu) = U(\Gamma_\mu, g) - U(\Gamma, g)$, *i.e.,* w_q when it derives a new subgoal q of g, and 0 otherwise. Write $V(S, g, b)$ for the best net total, earnings minus λ-weighted costs, that a chain from S can still collect with budget b left. V prices the future of a move, so the best first step earns the most, counting what follows it,

$$V(S, g, b) = \max\Big\{0,\ \max_{\mu}\ \big[\Delta U(S, g, \mu) - \lambda\, \text{cost}(\mu, S) + V(S_\mu, g, b - \text{cost}(\mu, S))\big]\Big\},$$

where the three terms are what a step earns, pays, and leaves to be earned, μ ranges over $\mathcal{A}(S)$ with $\text{cost}(\mu, S) \leq b$, and the leading 0 is the empty chain: the agent may take no step at all. At S_0 with $b = B$, this is the objective above, read per step. Both earlier objectives are special cases. With utility paid only for deriving g, maximizing V minimizes the cost of reaching g; with the chain stopped at the budget, it gathers utility, whether g is reached or not. Computing V exactly is intractable (Theorem 1).

Policy selection problem. We next state the problem.

Optimal Policy Selection Problem (OPSM). Given an initial state $S_0 = (G, \Sigma, \Gamma_0)$ and a goal g, the problem is to find a policy $\pi : (S, g) \to \mu$ that induces a reasoning chain C from S_0 that derives g, *i.e.,* with $g \in \Gamma_k$, at the minimum expected cost, *i.e.,* the cost-minimization objective above.

Budgeted Policy Selection Problem (BOSP). Given S_0, g and a budget B, find a policy π that maximizes the expected utility within the budget, *i.e.,* the utility-maximization objective above. For a random-

ized π, the expectation averages over the chains it induces, and the constraints bind each of them.

Example 5: Continuing Example 4, suppose two candidate motifs are available: $\mu_1 = (\varphi_1, h_1)$, which derives HighRisk(u), one application from g via (φ_2, h_2); and $\mu_2 = (\varphi_4, h_4)$, which derives Collusive(u, u') and can never support g. Under unit cost, the chain through μ_1 and (φ_2, h_2) reaches g at cost 2. *(1) OPSM.* An optimal policy derives g at the least cost: it applies (φ_1, h_1) and then (φ_2, h_2), and never opens the branch of (φ_4, h_4). *(2) BOSP.* With $B = 2$ the same chain completes. With $B = 1$ no chain can complete, and a budgeted policy spends the step where it brings the most utility toward g: on (φ_1, h_1), whose consequence is one application from g, rather than on (φ_4, h_4). □

3.3 Intractability

We settle the complexity of policy selection under OPSM and BOSP, in their decision versions: for OPSM, to decide, given (G, Σ, Γ_0, g) and a budget B, whether there exists a policy (equivalently, a reasoning chain) that derives g with $\text{cost}(C) \leq B$; for BOSP, to decide, given also a threshold η, whether there exists a policy with $\text{cost}(C) \leq B$ such that $U(\Gamma_{\leq B}, g) \geq \eta$.

Theorem 1: *The decision versions of OPSM and BOSP are both* NP*-complete.* □

Proof sketch: Both problems are in NP: a certificate is a reasoning chain $C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle$, and a shortest one is polynomially bounded: dropping failed applications, which derive nothing while costing at least 1, and repeated ones, which duplicate a step, yields a shorter valid certificate, failure-free and repeat-free, whose steps derive distinct new ground facts; GAR predicates have arity at most 2, hence $O(p \cdot |V|^2)$ steps, with p the predicates of Σ. We can check in PTIME that the preconditions of each φ_i hold under h_i *w.r.t.* the current fact set, simulate the derived facts, accumulate the cost, and check whether g is derived within B (OPSM) or whether $U(\Gamma_{\leq B}, g) \geq \eta$ with $\text{cost}(C) \leq B$ (BOSP).

The lower bounds are by reduction: OPSM from NP-complete Set Cover [20] and BOSP from budgeted maximum coverage [31], both by a gadget in which activating a set and covering an element each cost one unit, and the goal needs all elements covered (Appendix A). The hardness rests on shared subderivations being charged once: charged as trees, with shared premises paid anew, the minimum would be computable in polynomial time by a tropical fixpoint [32]. So OPSM and BOSP are NP-complete. □

This complements the NP-completeness of association deduction for GARs [17], which decides whether g is derivable at all: OPSM and BOSP ask for the cheapest and the most useful derivations.

3.4 Solution Scheme

In light of the intractability (Theorem 1), we propose a 3-step scheme for *approximate* policy selection: given a goal g, rules Σ, a graph G, initial facts Γ_0 and a budget B, it returns the chain built within B, which derives g whenever the budget suffices.

Static analysis. The first step analyzes the rules Σ and the goal g, once per goal and before any matching: it settles which predicates can lead to g at all, and how directly, an outcome that guides every step of the search.

Fact expansion. The second step adds facts to Γ, one at a time, in three stages. *Match.* When a new fact enters Γ, check only the pattern matches that it completes; each such match proposes a candidate motif (φ, h). A derived fact, *e.g.,* a Collusive edge, can also complete a pattern, so new candidates can appear as Γ grows. A pattern may propose many candidates, and a candidate may still fail. *Score.* Estimate cheaply how much each candidate would help reach g and what applying it would cost; this does not tell whether it will succeed. *Select and apply.* The policy π picks a scored candidate and tests its preconditions. If all hold, the consequent joins Γ; otherwise, nothing is derived and the failure is recorded.

Termination. The expansion repeats, charging each derivation against B, validated or not, and never deriving a candidate whose cost exceeds the remaining budget. When it stops depends on the objective. For optimal reasoning, deriving g does not end the search: it stops when no cheaper chain to g can still fit in what remains of B, and returns the cheapest chain found. For anytime reasoning, it stops when B runs out and returns the chain built. Either way, the chain returned costs at most B by construction.

4 SCORING MOTIFS

This section scores each candidate motif by two estimates: how much it would earn toward g (*estimated utility*), and how much resource it would consume (*estimated cost*). We first identify what makes scoring hard, and then design the score to meet these difficulties.

Problem. Given a state $S = (G, \Sigma, \Gamma)$, a goal g, and a candidate motif $\mu = (\varphi, h)$, the task is to score μ, approximating $\Delta U(S, g, \mu)$, the utility the step would add toward g, and the cost it would pay.

Challenges. Three challenges stand out.

[C1] Goal relevance. At each step, many motifs are applicable: some carry the search straight toward g, some take detours, and some derive facts that can never reach g at all. The score must grade this: the more directly a candidate leads to g, the higher it scores, and one that can never reach g scores zero.

[C2] Symbol grounding. Scoring a candidate draws on both of its parts: the rule settles whether it can lead to g at all, and the match whether its premises hold. The two are of entirely different kinds: a rule is a symbolic condition over variables, a match a data-level assignment over G. The hard part is to combine the two into a single score, at every step, *i.e.,* to ground the symbolic conditions in G [25].

[C3] Informedness vs. cost. The score sits on the decision path: at each step, every candidate is scored before one is applied. Scoring thus has to be cheap.

4.1 The Scoring Function

In light of the challenges, we guide the exploration with a scoring method. The two quantities that matter for a step, its marginal utility $\Delta U(S, g, \mu)$ and its actual cost $\text{cost}(\mu, S)$ (Section 3.2), are known only after μ is applied; the score estimates both from the current state. At each step, given a state $S = (G, \Sigma, \Gamma)$, the goal g and a candidate motif μ, the agent scores μ by

$$f(S, g, \mu) = \alpha \cdot \text{rel}(S, g, \mu) - \beta \cdot \widehat{\text{cost}}(\mu, S),$$

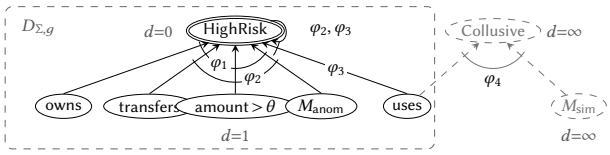


Figure 4: The dependency graph $D_{\Sigma,g}$ of Example 2; an arc marks a rule (AND), and $d(\cdot)$ the backward distance from HighRisk.

where $\text{rel}(S, g, \mu)$ estimates the marginal utility $\Delta U(S, g, \mu)$ the step would earn, and $\widehat{\text{cost}}(\mu, S)$ the actual cost $\text{cost}(\mu, S)$ it would pay; $\alpha, \beta \geq 0$ balance the two.

4.1.1 Estimated Utility. A step earns utility only when it succeeds and derives a new fact that serves g. Its expected utility thus has two factors: what the new fact is worth toward g, and whether the step succeeds at all. We estimate the two from two sources. *Scoring from the rules*, rel_{fam} estimates the worth from a dependency graph of Σ: it tells whether the rule of a candidate can lead to g, and how directly. *Scoring from the facts*, rel_{evd} estimates the success from the facts witnessed on the current chain: the more premises of a candidate are among them, the more likely it succeeds, and with all premises witnessed it succeeds for sure. We develop the two in turn, and then combine them into a single utility measure.

Scoring from the rules. The utility of a candidate is first read off its rule.

The dependency graph. We first organize the rules of Σ, once per goal, into a data structure that the score will consult, the *dependency graph* $D_{\Sigma,g}$ (Figure 4). It has as nodes the predicates that can derive g, directly or indirectly, and an edge $q \to p$ whenever a rule φ uses q to derive p, labeled by φ; the same pair witnessed by several rules is kept once, carrying the least weight when the edges are weighted for scoring (see below). Edges compose into paths, and a path from q to g is a chain of rule applications that derives g from q, its length counting the applications. The graph is predicate-level, fixed by Σ and g alone, whatever the instance at hand.

Scoring. The static analysis prices each predicate of rule of motif by its distance to g. Let $d(q)$ be the fewest rule applications by which q can support g, *i.e.,* the length of the shortest path from q to g in $D_{\Sigma,g}$, with $d(q) = \infty$ if q is absent from $D_{\Sigma,g}$. (With edges weighted by w_0, $d(q)$ lower-bounds the cost of supporting g instead. The distance is predicate-level, blind to instances: when the goal's predicate already has instances, the price stays a valid lower bound but may be loose.) The score of a candidate $\mu = (\varphi, h)$ sums these distances over the predicates of its rule φ:

$$\text{rel}_{\text{fam}}(\mu) = \gamma^{\sum_{q \in \varphi} d(q)},$$

where the sum ranges over the *distinct* predicates of φ: pattern edges, attribute and ML predicates and the consequent alike, each counted once however often it occurs; and $\gamma \in (0, 1)$ discounts each application still ahead. A rule serves g only when all its predicates do; one absent from $D_{\Sigma,g}$ scores the candidate 0, soft filtering.

$D_{\Sigma,g}$ and $d(\cdot)$ are built once per goal by a backward shortest-path computation over Σ, a BFS under unit cost; at runtime, the score of a candidate is a single lookup of $d(\cdot)$ per predicate.

Example 6: Take the rules of Example 2 and $g = \text{HighRisk}(v)$ (Figure 4). Under unit cost, the distances are $d = 0$ for HighRisk, and $d = 1$ for owns, transfers, amount $> \theta$, M_{anom} and uses, the premises of φ_1, φ_2 and φ_3; Collusive and M_{sim} are absent from $D_{\Sigma,g}$ ($d = \infty$). With $\gamma = 1/2$, a candidate of φ_3 scores $\gamma = 1/2$ (its distinct predicates uses and HighRisk at 1, 0), one of φ_2 scores $\gamma^3 = 1/8$ (owns, transfers, amount $> \theta$ and HighRisk at 1, 1, 1, 0; owns occurs twice in the pattern and counts once), one of φ_1 scores $\gamma^4 = 1/16$, and one of φ_4 scores 0 and is filtered out. The rules alone thus rank φ_3 above φ_2; Example 7 shows how the facts reverse this. □

Scoring from the facts. Progress toward g is made of facts: a candidate advances the chain only by deriving a new fact that serves g, and it derives one only when its premises hold. The facts at hand tell whether this will happen; the rules cannot, since rel_{fam} scores all candidates of the same rule alike. We therefore score the candidate against the state: the more of its premises are already witnessed on the current reasoning chain, *i.e.,* matched or derived by its earlier steps, the more likely it derives the fact and the chain advances toward g. The reading compares symbolic premises with data-level facts, two different kinds (C2). Hyperdimensional computing (HDC) [29] puts both in one space.

Hyperdimensional computing. We recall HDC [29]: each symbol carries a random bipolar vector of dimension *e.g.,* $D \approx 10^4$, two distinct symbols nearly orthogonal with high probability. Vectors compose by *bundling* $\oplus$ (componentwise sum): the sum remains similar to each summand, and dissimilar to any vector not in the sum. Positions are encoded by a fixed *permutation* ρ, a cyclic shift: $\rho(v)$ and v itself are nearly orthogonal. Similarity of vectors are measured by the cosine.

Encoding facts. We associate a random vector v_p with each predicate p, and a vector v_a with each vertex a of G, and represent a fact by bundling the vectors of its predicate and its vertices: HighRisk(u) is $v_{\text{HighRisk}} \oplus v_u$. To tell the positions of the vertices apart, the i-th vertex is shifted before it joins the bundle: $v_f = v_p \oplus \bigoplus_{i=1}^{k} \rho^i(v_{a_i})$, normalized to unit length. The shift distinguishes *e.g.,* transfers(a, a') from transfers(a', a), which bundling alone would identify; the normalization gives every fact equal weight in a bundle, whatever its arity.

The evidence vector. The score must tell, for every candidate and at every step, how many of its premises are already witnessed; computed from scratch, this would cost more than the deduction it guides (C3). We therefore keep the facts witnessed on the reasoning chain in a single vector, and read it with one dot product. The facts *witnessed* on the current reasoning chain are bundled into $E = \bigoplus_f v_f$, the *evidence vector*: one summand v_f per fact, encoded as above, where f ranges over the pattern facts the chain's applications matched, successful or failed, plus the facts its successful applications derived; a match computed is information gained, even when the remaining preconditions fail. E grows by one vector addition per witnessed fact. It covers the chain built for g rather than the entire fact set Γ, starts at 0 on a fresh chain (where $\text{rel}_{\text{evd}} = 0$ by convention), and is carried per branch in the branch-and-bound. The closer the bundled premises of a candidate are to E, the more of them are already witnessed, and the more likely the candidate derives a new fact, *i.e.,* makes real progress toward g.

For a candidate $\mu = (\varphi, h)$, we first ground each premise $p(\bar{x})$

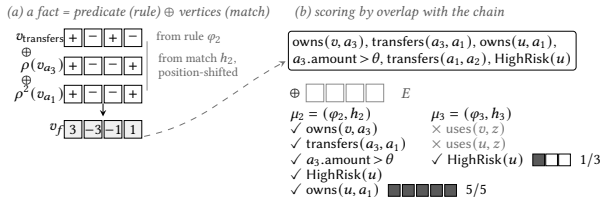


Figure 5: HDC scoring of Example 7: (a) facts bundle predicates and shifted vertices; (b) candidates score by cosine with E.

of φ under h and bundle the grounded fact vectors into a single *premise vector* $v_\mu = \bigoplus_{p(\bar{x})\in\varphi} v_{p(h(\bar{x}))}$; $\mathrm{rel}_{\mathrm{evd}}(\mu)$ is then its cosine with E:

$$\mathrm{rel}_{\mathrm{evd}}(\mu) = \cos\left(v_\mu,\ E\right).$$

By near-orthogonality, each premise fact already witnessed by the chain contributes $\approx +1$ to the inner product, and each absent one ≈ 0; the cosine normalizes the count by the two bundle sizes (bounded in Appendix I). It is one dot product per candidate, in $O(D)$ time and free of matching; and as E grows at each step, the signal evolves with the state.

Putting the two together. The two signals combine into the utility measure:

$$\mathrm{rel}(S, g, \mu) \ = \ \mathrm{rel}_{\mathrm{fam}}(\mu) \ + \ \nu \cdot \mathrm{rel}_{\mathrm{evd}}(\mu),$$

with $\nu \geq 0$ balancing the two: $\mathrm{rel}_{\mathrm{fam}}$ is read off the fixed space and $\mathrm{rel}_{\mathrm{evd}}$ off the evolving state. The sum estimates $\Delta U(S, g, \mu)$, which pays w_q when the step derives a new subgoal q of g (Section 3.2). With $w_q = \gamma^{d(q)}$, the consequent of μ carries $\gamma^{d(p_0)}$, priced by $\mathrm{rel}_{\mathrm{fam}}$; the derivation goes through only when the premises hold, read by $\mathrm{rel}_{\mathrm{evd}}$; and the fact is new for free, since $\mathcal{A}(S)$ keeps out what Γ already has. The premise distances summed in $\mathrm{rel}_{\mathrm{fam}}$ have no counterpart in ΔU; they tilt the estimate toward rules closer to g, a bias priced in the ϵ of Theorem 6.

Example 7: Continue Example 6, with $D = 4$ for illustration (Figure 5). The symbols carry random vectors, *e.g.*, $v_{\mathrm{transfers}} = (+,-,+,-)$, $v_{a_3} = (+,+,-,-)$ and $v_{a_1} = (-,+,+,-)$, and ρ shifts each coordinate cyclically left by one; the fact $\mathrm{transfers}(a_3, a_1)$ is thus $v_{\mathrm{transfers}} \oplus \rho(v_{a_3}) \oplus \rho^2(v_{a_1}) = (3,-3,-1,1)$. Suppose that, probing how to derive g, the chain first probed (φ_2, h_2): the probe matched its pattern facts $\mathrm{owns}(v, a_3)$, $\mathrm{transfers}(a_3, a_1)$ and $\mathrm{owns}(u, a_1)$, and computed $a_3.\mathrm{amount} > \theta$ — witnessed by the evaluation itself — but failed on $\mathrm{HighRisk}(u)$; the chain then applied (φ_1, h_1), whose match touched $\mathrm{owns}(u, a_1)$ and $\mathrm{transfers}(a_1, a_2)$, and derived $\mathrm{HighRisk}(u)$; E sums the witnessed facts, the failed probe's among them. Consider $\mu_2 = (\varphi_2, h_2)$ and $\mu_3 = (\varphi_3, h_3)$, both deriving g (Figure 3). Scoring from the rules ranks μ_3 above μ_2 (γ against γ^3, Example 6). Scoring from the facts reverses this: all five premises of μ_2 are summands of E, while of the three premises of μ_3, only $\mathrm{HighRisk}(u)$ is witnessed. Reading the counts as cosines (Appendix I), E bundles six facts, and μ_2 reads $5/\sqrt{5 \cdot 6} \approx 0.91$ against $1/\sqrt{3 \cdot 6} \approx 0.24$ for μ_3: at the defaults $\gamma = 1/2$ and $\nu = 1$, $\mathrm{rel}(\mu_2) = 1/8 + 0.91 \approx 1.04$ exceeds $\mathrm{rel}(\mu_3) = 1/2 + 0.24 \approx 0.74$. The failed probe was not wasted: its match now ranks μ_2 first, and with $\mathrm{HighRisk}(u)$ derived, μ_2 will succeed. □

4.1.2 Estimated Cost. The second estimate, $\widehat{\mathrm{cost}}(\mu, S)$, prices the application of a candidate $\mu = (\varphi, h)$, *i.e.*, the evaluation of the preconditions of φ under h, estimated before any of them runs; execution charges the actual cost against the budget, whether they hold or not.

The cost of an application. Not every precondition needs evaluating: one whose grounded fact is already witnessed on the chain has its result on record, and costs a lookup only. The evidence vector E tells the two apart, premise by premise. Define the *witnessed flag* $s_p \in \{0, 1\}$ of a precondition p: $s_p = 1$ iff the grounded fact $p(h(\bar{x}))$ is already a summand of E, read off by cosine, $\cos(v_{p(h(\bar{x}))}, E) \geq \tau$ for a fixed threshold τ; by near-orthogonality the read is near $1/\sqrt{|E|}$ for a summand and near 0 otherwise, so a threshold separates the two cases. The estimate sums over the preconditions still untested:

$$\widehat{\mathrm{cost}}(\mu, S) = \sum_p (1 - s_p) \cdot T(p),$$

where p ranges over the preconditions of φ and $T(p)$ is the estimated cost of evaluating p, whatever the grounding. This is the same per-premise read of E that $\mathrm{rel}_{\mathrm{evd}}$ uses: one evidence vector prices both what a step would earn and what it would pay.

A cost model from the log. The function T is a *cost model*, read off a log. Execution keeps a log of predicate evaluations over the current and earlier reasoning chains: each record pairs a predicate, identified by its vector v_p, with the observed cost of one evaluation, and $T(p)$ is the average of these observations. Costs are measured in wall time; under unit cost each evaluation costs 1 and the budget counts reasoning steps. Without records, $T(p)$ falls back to a static measure $h(p)$, which grows with the number of variables p reads and the size of their attribute vectors, scaled to the log by the average ratio of observed costs to static measures over the predicates logged so far.

5 APPROXIMATING POLICY SELECTION

This section develops practical strategies to approximate policy selection: policies that follow the score, deterministic or randomized; policies that learn it from past reasoning chains; policies that guard it with logic; and execution that steers the search by the budget.

Problem. Given an initial state $S_0 = (G, \Sigma, \Gamma_0)$, a goal g, a budget B and the scoring function f, the task is to find and execute a policy π that, guided by f, selects a motif at each step so that its induced chain approaches the value $V(S_0, g)$, optimal when the budget suffices and anytime when it does not. Exact selection is NP-complete (Theorem 1), so π must approximate, deciding each step by the score.

5.1 Deterministic and Randomized Policies

We start with policies that follow the score: deterministic ones commit to its top choice, and randomized ones sample accordingly.

Deterministic policies. A deterministic policy follows the score: at each state and goal, it evaluates the applicable motifs by their scores and chooses a best one, greedy with respect to the score. Formally, it approximates OPSM by

$$\pi(S, g) = \arg\max_{\mu \in \mathcal{A}(S)} f(S, g, \mu),$$

where f is the shared scoring function.

At each step, incremental matching proposes candidate motifs

(φ, h): the agent scores each by f, and derives a highest-scoring one, extending the chain when its preconditions all hold. The loop repeats until g is derived, no candidate remains, or the budget is exhausted, committing to each choice: a branch that derives nothing still costs budget.

Randomized policies. The score is only an estimate, and committing to it makes its errors permanent: a branch the score underrates is never visited, and the same error recurs in every run. We develop randomized policies to improve the success rate, *i.e.,* the probability that a run derives g within the budget: the motif is drawn with probability increasing with the score,

$$\pi(S, g)(\mu) = \frac{\exp(f(S, g, \mu)/\tau)}{\sum_{\mu' \in \mathcal{A}(S)} \exp(f(S, g, \mu')/\tau)},$$

where f is the shared scoring function and $\tau > 0$ a configurable temperature: low τ approaches the deterministic policy, and high τ spreads the budget over branches.

5.2 Learned Policies

We next develop learned policies to improve the score itself. The handcrafted score is fixed: relevance from rule text and premise overlap, cost from logged averages. Past reasoning chains carry what it misses: the rules settle which motifs *can* derive g, and the chains record which ones *did*, and at what cost.

Modeling and selection. A learned policy is parameterized by a model π_θ that scores each candidate motif and normalizes the scores into a distribution:

$$\pi_\theta(S, g)(\mu) = \mathrm{softmax}\big(f_\theta(S, g, \mu)\big),$$

where f_θ is the learned score and π_θ the induced distribution over the applicable motifs.

Feature construction. The input $\phi(S, g, \mu)$ to f_θ assembles what the state already computes. Per candidate $\mu = (\varphi, h)$, it concatenates (1) the vector v_μ of the candidate, its premises and consequent instantiated by h and bundled by the HDC scheme; (2) the evidence vector E of the chain so far and the vector v_g of the goal by the same scheme; (3) the distances of its premise predicates from g in $D_{\Sigma,g}$, as a profile $(s_0, s_1, \dots)$, s_k reading off those k rule applications from g; and (4) scalars: $\mathrm{rel}_{\mathsf{fam}}(\mu)$, $\mathrm{rel}_{\mathsf{evd}}(\mu)$ and $\widehat{\mathrm{cost}}(\mu, S)$, the number of premises already in Γ, the remaining budget, and the confidence of ML predicates. The model f_θ is a two-layer perceptron over these features.

Learning. The model f_θ is trained offline and applied online. Offline, it learns from reasoning chains recorded from real runs by the policies above, each labeled by execution itself with the cost it paid and the utility it gained. Online, at each step the agent encodes every applicable motif as $\phi(S, g, \mu)$, feeds it to the model, and ranks the candidates by its learned scores.

Supervised learning. Each recorded chain that derived its goal contributes one positive sample $(\phi(S_{t-1}, g, \mu_t), 1)$ per step, and each motif applicable at the same state but passed by a hard negative $(\phi(S_{t-1}, g, \mu^-), 0)$: it looks plausible, yet the chain passed it by [28]. Such negatives are noisy, since a motif passed by may derive g as well; the loss below asks only that the recorded choice outscore them, not that they be useless. Nor are failed chains wasted: one that missed its goal still derived facts, and is relabeled as a positive trace for each of them [3]. The score is trained on these pairs by a logistic ranking loss,

$$L_{\mathrm{sup}} = \sum_{(\mu_t, \mu^-)} \log\big(1 + \exp\big(f_\theta(S_{t-1}, g, \mu^-) - f_\theta(S_{t-1}, g, \mu_t)\big)\big),$$

one term per recorded motif μ_t and a negative μ^- at the same state.

Reinforcement learning. The model is trained in episodes of self-play. In each episode the policy plays out a reasoning chain, sampling an applicable motif by its current scores at each step, until it derives the goal or the budget runs out. The chain is scored by its return $R(C)$, its utility minus its total cost, and each step has its probability pushed up or down in proportion to the return that followed it, by policy gradient [47], which maximizes the expected return over the chains the policy induces,

$$J(\theta) = \mathbb{E}_{C \sim \pi_\theta}\big[R(C)\big].$$

Recorded runs provide episodes for free, but price only the steps they took; fresh episodes played offline price any candidate, taken or not, by running the policy to the end a constant number of times and averaging the returns. The exploration is paid at training time; once trained, the model scores a candidate online by one forward pass. The price is credit assignment: one final return must be attributed to the steps of a long chain.

Example 8: Continuing Example 4, suppose a recorded run for goal HighRisk(v) applied (φ_1, h_1) and then (φ_2, h_2), at costs 2 and 3, and derived the goal, of utility 10. The two chosen motifs are positive samples, and (φ_4, h_4), applicable at the initial state but passed by, a negative one; the chain is also a positive trace for goal HighRisk(u), derived on the way. The two steps differ in value, the first leading to a return of $10 - 2 - 3 = 5$ and the second to $10 - 3 = 7$, a grading that the binary labels do not carry. A score learned from many such chains ranks motifs on productive routes high, even where the handcrafted f cannot tell them apart. □

5.3 Hybrid Neuro-Symbolic Policies

We develop hybrid policies to guard the model's score with logic: rule out the provably useless by logic, and rank the rest by the model.

Modeling and selection. Formally, a hybrid policy maximizes the model's score over the admissible candidates,

$$\pi(S, g) = \arg\max_{\mu \in \mathcal{A}(S) \cap C(S)} f_\theta(S, g, \mu),$$

where $C(S)$ denotes the logically admissible motifs: it keeps those that (a) can reach g, *i.e.,* $d(p_0) < \infty$, and (b) derive something new, *i.e.,* the consequent fact is absent from Γ; applicability comes for free from matching.

Symbolic filtering. The filter drops every rule whose consequent cannot reach g, and nothing else, since $d(p_0) = \infty$ if and only if no chain of rules leads from p_0 to g; deduction thus stays complete (Corollary 4). This is the hard form of the soft filtering of $\mathrm{rel}_{\mathsf{fam}}$, over the same $d(\cdot)$, settled once per goal by one lookup per rule: rules that cannot reach g are never matched at all, and the survivors are then ranked by the model, since $d(p_0) < \infty$ promises no grounded derivation in G.

LLM-assisted guidance. The score f_θ need not be learned from chains at all. A large language model suggests candidate motifs

Algorithm HybNS
Input: graph G, rules Σ, initial facts Γ_0, goal g with $D_{\Sigma,g}$ and $d(\cdot)$, budget B, model f_θ.
Output: the chain C^* found and its cost $\hat{C}$, optimal when line 6 keeps all candidates, or an anytime chain when no chain within B exists.

1. $\hat{C} \leftarrow +\infty$; $C^* \leftarrow$ none; $F \leftarrow \{(\Gamma_0, 0, \emptyset)\}$;
2. **while** $F \neq \emptyset$ **do**
3. $(\Gamma, c, \mathcal{R}) \leftarrow$ a node of F with the smallest $c + \min_\mu \text{price}(\mu)$;
4. $C \leftarrow$ motifs μ applicable at Γ, deriving new facts, with $d(p_0) < \infty$, $c + \text{price}(\mu) \leq B$, $c + \text{price}(\mu) < \hat{C}$, and not retired in $\mathcal{R}$ at Γ;
5. **if** $C = \emptyset$ discard the node and continue;
6. score C by f_θ, keep the top-k, and re-rank by price, ties by rel;
7. apply the best μ, charging its actual cost c_μ;
8. **if** some precondition fails, record $(\mu, \Gamma, \text{static})$ in $\mathcal{R}$ when the failed preconditions read G alone, else $(\mu, \Gamma, \text{dynamic})$; push $(\Gamma, c{+}c_\mu, \mathcal{R})$ and $(\Gamma, c, \mathcal{R})$ to F;
9. **else**
10. $\Gamma' \leftarrow \Gamma \cup \{p_0(h(\bar{x}_0))\}$; $c' \leftarrow c + c_\mu$;
11. **if** $g \in \Gamma'$ and $c' \leq B$: $C^* \leftarrow$ the chain to Γ'; $\hat{C} \leftarrow \min(\hat{C}, c')$;
12. **else if** $c' \leq B$ push $(\Gamma', c', \mathcal{R})$ to F;
13. push $(\Gamma, c, \mathcal{R} \cup \{(\mu, \Gamma, \text{static})\})$ to F;
14. **if** C^* is none, extend the partial chain of the highest utility: greedily apply admissible best-rel motifs deriving new facts while its cost stays within B;
15. **return** C^* and $\hat{C}$ (and the anytime chain when C^* is none).

Figure 6: Hybrid neuro-symbolic reasoning as branch and bound; price(μ) lower-bounds completing an irredundant chain through μ.

or reasoning steps, which are then verified and refined by logical checks. The prompt presents the goal g, the facts at hand and the candidates of $\mathcal{A}(S)$, each with its rule and premises, those already in Γ marked; the model is asked to rank, not to invent. A decision model such as Jev [52] answers in one pass with a calibrated distribution over the candidates, which serves as π directly, constrained to the candidate set by construction [34]. Its three decision primitives each find a use: *choice*, picking one of k given options, is the policy itself; *score*, grading an option on an ordered rubric, feeds the re-ranking at execution; and *noul*, the probability of a yes/no proposition, prices *e.g.,* whether a candidate is worth matching at all, a soft gate screening a large candidate set down to a choice. The calls are batched and capped per step: at most K screen calls over the top-$K \cdot L$ candidates by the current score f, then one *choice* call closes; the rest are ranked by f alone. Like all scoring, they are control overhead, and B meters only the chain's applications. Refinement and verification stay with logic: a suggested rule is grounded into candidates by matching, and each is checked for applicability, admissibility ($d(p_0) < \infty$), novelty and budget (price); one that fails is dropped, and when none survives, the policy falls back to f.

5.4 Execution and Adaptation

We next study how the agent executes and adapts policies to approximate optimal and anytime reasoning in practice; which applies is revealed only as the search proceeds.

Algorithm. The algorithm is a best-first branch and bound search (Figure 6), in two regimes.

(1) Optimal reasoning. The search keeps a set of open branches, and the best chain found so far, C^* with its cost $\hat{C}$ (line 1). At each round it expands the branch that looks cheapest to complete, by the cost so far plus the least completion price of its candidates (line 3). Of the branch's candidates it keeps only those that can still beat C^* within the budget and are not retired at its fact set (lines 4-5), takes the top-k among them by the model, and applies the cheapest to complete (lines 6-7). A failed application pays on the branch that attempted it: the search pushes the charged continuation and, unpaid, the sibling that skips the motif, recording the failure against the current fact set — for good when the failed preconditions read G alone, and only against that fact set otherwise, so the motif is re-admitted once the failed premise's predicate has grown (line 8). A successful application within the budget pushes the new state and the sibling of untried alternatives; one that overshoots keeps only the alternatives (lines 10-13). Reaching g within the budget records the chain as C^* and tightens $\hat{C}$ (line 11), and the loop runs until no branch survives (line 2). C^* is then optimal, provided the score discarded no candidate at line 6 (Theorem 5).

(2) Anytime reasoning. If no chain was found, the bounds have certified that none can fit the budget; the agent then extends the partial chain of the highest utility, greedily applying an admissible motif of the highest relevance at each step while its budget remains (line 14). Under a hard wall-clock cutoff, the search stops and returns the best recorded chain, or the greedy partial chain of the highest-utility frontier branch when none is recorded: B certifies the chains, and the cutoff bounds only the search.

Branch and bound. Why does the search end with the optimal chain, or with a proof that none fits the budget? The two bounds do the work. Write $\text{price}(\mu) = w_0(\varphi) + d(p_0)$ for the *completion price* of a candidate: its own application, certified below by w_0 (Section 3.2), plus the applications still ahead, lower-bounded by $d(p_0)$ with the edges of $D_{\Sigma,g}$ weighted by w_0. Along an irredundant chain, one in which every step derives a fact that is used, the suffix from a candidate projects to a walk from its consequent p_0 to g in $D_{\Sigma,g}$, so the completion price lower-bounds completing through the candidate; a branch cut by it thus completes an irredundant chain only over budget or without improving on the recorded chain, and a redundant chain is dominated by its irredundant subchain, which a sibling branch explores at strictly smaller cost. Retiring a success on the skip-sibling loses nothing either: a successful application costs the same at any state where its preconditions hold, so a chain that applies the motif later is simulated by the success child with the motif first. Expanding cheapest first brings the first chain, and with it $\hat{C}$, early, and each chain recorded only tightens the pruning; with exact costs this is A*, and each test is one lookup of $d(\cdot)$. Two things keep it exact: the bounds are certified below by w_0, and line 6 drops no candidate that may still win; with a bounded top-k the search trades the guarantee for speed, since a candidate cut by the score may hide the cheapest chain, or the only one, and "none recorded" no longer certifies that none fits the budget.

Constrained re-ranking. Re-ranking serves anytime reasoning. When the search proves that no chain can reach g within the budget, the agent changes objective, from reaching g at the least cost to collecting the most utility before the budget runs out. A single

greedy pass then applies, at each state,

$$\pi(S, g) = \arg \max_{\mu \in \mathcal{A}(S) \cap C(S)} \mathrm{rel}(S, g, \mu),$$

while the budget remains (line 14). Greedy suffices here, since with no completion to price, a step earns only its own marginal utility; and the criterion is the utility term of the score, since with the budget fixed, cost turns from a criterion into a constraint. Marginal utility is known only after the step is applied, so the pass ranks candidates by its estimate, rel: $\mathrm{rel}_{\mathrm{fam}}$ tells how much the derived fact would be worth toward g, and $\mathrm{rel}_{\mathrm{evd}}$ whether its premises are witnessed on this chain — premises in Γ_0 or derived on sibling branches may be in place yet unwitnessed — hence whether the step is likely to derive at all, since a failed step earns nothing. Under unit cost this is the pass that Theorem 6 bounds; when costs vary, the criterion reads rel per unit cost.

Example 9: Continue Example 4: the bank now escalates flagged high-risk customers to a blacklist, via a rule φ_5 whose pattern is a flag fact flagged(x) with precondition HighRisk(x), deriving Blacklist(x); Γ_0 flags v alone, and also carries the pattern of u for another customer w; and the goal is g = Blacklist(v). Hence d(Blacklist) = 0, d(HighRisk) = 1, and Collusive cannot reach g in $D_{\Sigma,g}$. With a budget of five, the filter drops the model's preference (φ_4, h_4) by a lookup, and price speaks first: (φ_5, v) completes at $1 + d(\mathrm{Blacklist}) = 1$, cheapest on the board, and is tried at the root, a false start, HighRisk(v) yet underived, paid on the charged branch. The unpaid sibling, its evidence empty, breaks the price-2 tie by $\mathrm{rel}_{\mathrm{fam}}$: (φ_3, h_3) (γ^3) is tried and fails, then (φ_2, h_2) (γ^7), two more false starts, each costing one unit on its own charged branch while the unpaid siblings drive on untouched, before (φ_1, h_1) derives HighRisk(u). There (φ_5, v), re-admitted now that the failed premise's predicate has grown, is tried again at price 1 and fails again; the next unpaid sibling continues with (φ_2, h_2), whose premises now stand on the chain, derives HighRisk(v), and (φ_5, v), re-admitted once more, closes at the next step, a chain of three. The line for w, explored on a sibling, derives HighRisk(w) and dies short of g with the rest: every branch still open when $\hat{C} = 3$ lands is cut at one lookup each; the recorded chain is certified optimal, the false starts paid only by the branches that made them. With a budget of two, the false starts go as before, each failure paid only on its charged branch, and (φ_1, h_1) derives HighRisk(u); (φ_5, v), re-admitted, is tried once more at $1 + 1 = 2$, exactly the budget, and fails; every other candidate completes at $1 + 2 = 3$, over budget, and no chain is recorded, certifying that g is unreachable in two steps. A greedy pass by relevance then spends the last step on (φ_2, h_2), whose premise HighRisk(u) just landed on the chain, deriving HighRisk(v) and stopping one short of g with an informative partial chain. □

Complexity and adaptation. Each expansion scores the candidates of one branch in $O(|\mathcal{A}(S)| \cdot T_{\mathrm{model}})$, applies one in T_{eval}, and prunes by lookups of $d(\cdot)$; the recorded chain prunes the expansions in practice. Across workloads, the symbolic layer transfers exactly: a new goal asks for one backward BFS to rebuild $D_{\Sigma,g}$ and $d(\cdot)$, and the learned score is retrained on chains of the new workload.

6 THEORETICAL GUARANTEES

6.1 Correctness Guarantees

Soundness. We first show that policy-guided deduction preserves the logical correctness of GAR semantics (the chase).

Theorem 2: *Let Γ_0 be a set of valid initial facts over G, and Σ a set of* GARs. *For any policy π, every fact derived along a reasoning chain induced by π is logically entailed by (G, Σ, Γ_0). That is, for any state $S_k = (G, \Sigma, \Gamma_k)$ reached by π, $\Gamma_k \subseteq \Gamma^* = \mathsf{Chase}(G, \Sigma, \Gamma_0)$.* □

Proof sketch: Only a successful application derives, and one is a chase step, deriving a fact entailed by the current facts and Σ; a failed one adds nothing. Since the chase computes the closure of all such valid applications, every derived fact is in Γ^* (details in the appendix). □

Goal-level Completeness. **Theorem 3:** *Let $\Gamma^* = \mathsf{Chase}(G, \Sigma, \Gamma_0)$ be the chase closure. If a goal $g \in \Gamma^*$, then there exists a policy π such that the reasoning chain induced by π derives g. Moreover, if π is* fair, i.e.,*it never permanently ignores a motif whose preconditions may later hold: it retires one that fails on static preconditions, which no growth of Γ can make hold, and re-admits one that fails on dynamic preconditions once Γ grows, and candidates that become proposable as Γ grows, a derived fact completing their pattern, enter $\mathcal{A}(S)$ then and are quantified over from that point on, then π derives g whenever $g \in \Gamma^*$.* □

Proof sketch: The proof is in the appendix: some finite chase sequence derives g from Γ_0, and a fair policy tries every motif of it when its turn comes, re-admitting dynamic failures as Γ grows. □

Corollary 4: *Filtering by the dependency graph preserves completeness: a policy that never selects motifs whose consequent p_0 has $d(p_0) = \infty$ still derives g whenever $g \in \Gamma^*$, if it is fair over the surviving motifs.* □

Proof sketch: If $g \in \Gamma^*$, some chase sequence derives g; take the minimal subsequence of it that still derives g: on it every derived fact is used, so every consequent reaches g through the suffix, *i.e.,* $d(p_0) < \infty$, and the subsequence survives the filter; Theorem 3 then applies over the survivors. □

6.2 Performance Guarantees

Optimal reasoning chains. A greedy policy by itself carries no worst-case guarantee: the score is an estimate, and a chain descending it may depart arbitrarily from the cheapest. The guarantee comes from the branch and bound of Figure 6, which prunes only by certified bounds.

Theorem 5: *Let C^* and $\hat{C}$ be the chain and its cost returned by the search of Figure 6, run with the certified weights w_0 and without top-k truncation at line 6. If a chain deriving g within budget B exists, then C^* derives g at cost $\hat{C} \le B$, and no chain deriving g within B costs less than $\hat{C}$: C^* is optimal within B; if none exists, no chain is recorded.* □

Proof sketch: The proof is in Appendix E: the completion price certifies that a branch cut by it completes an irredundant chain only over budget or without improving on the recorded chain, a redundant chain is dominated by its irredundant subchain, which a sibling branch explores at strictly smaller cost, and a dynamic failure bars a motif only at the fact set that saw it fail. □

Table 1: Datasets ($K = 10^3$, $M = 10^6$, $B = 10^9$)

Dataset	Domain	#vertices	#edges	#relations	avg. deg.
FB15k-237	encyclopedic	14.5K	272K	237	18.7
ogbl-biokg	biomedical	93.8K	4.76M	51	50.8
FinReflectKG	financial	1.0M	17.5M	3.0K	17.5
DBpedia	encyclopedic	5.0M	20M+	600	4.0
Synthetic	synthetic	100M	1B	20	10.0

Guarantees for anytime reasoning chains. For anytime reasoning the budget binds with the goal out of reach, and the greedy pass banks utility step by step. Three properties make the pass amenable to analysis: under unit cost, the remaining budget is a cardinality constraint of B' steps; the utility is a weighted coverage of subgoals, monotone and submodular; and rel, by which the pass ranks, estimates the marginal utility of a step, with error at most ϵ, *i.e.,* the chosen motif's true marginal utility is at most ϵ below the largest available.

Theorem 6: *Assume unit cost, a monotone and submodular utility U, and per-step estimates within ϵ as above. Start the greedy pass at a state S with remaining budget B', and assume that every motif of the optimal continuation stays available to the pass, as holds when all preconditions are static over G. Then the utility gathered satisfies*

$$U(C_\pi) \geq (1 - 1/e) \cdot U(C^*) - B'\epsilon,$$

where C^* is the utility-optimal continuation from S within B'. □

Proof sketch: The proof is in Appendix D: averaging over the applications of C^* closes a $1/B'$ fraction of the remaining gap at each step, minus ϵ; unfolding the recurrence yields the $(1 - 1/e)$ factor of [39] with an additive ϵ per step. □

Two idealizations enter, and both are priced. Availability is a hypothesis, met when all preconditions are static over G; with derived preconditions a motif of C^* may be unavailable when the pass would pick it, and the bound is approximate. When costs vary, the budget is no longer a cardinality constraint, and ranking by marginal utility per unit cost restores a constant-factor guarantee at the loss of a constant [31]. The pass also pays for failed steps: ϵ bounds the error of every pick, including one whose preconditions fail, whose true marginal utility is 0 — a large-ϵ event, absorbed in ϵ.

6.3 Convergence and Bounded Cost

We also study whether policy-guided deduction terminates.

Theorem 7: *For a finite graph G and a finite rule set Σ, any reasoning chain induced by a policy π terminates in finitely many steps; under a budget B, every chain it returns moreover costs at most B.* □

Proof sketch: The proof is in the appendix: each step adds a new fact to Γ or retires a candidate, each motif is tried at most once per distinct fact set, and both pools are finite; under a budget, execution applies a motif only when its cost keeps the chain within B. □

Its runtime and its convergence to the chase closure under fair exploration are bounded in the appendix (Theorems 8 and 9).

7 EXPERIMENTAL STUDY

Experimental setting. We start with the experimental setting.

Datasets. We used four real-life knowledge graphs in a scale ladder from 0.27M to over 20M triples, summarized in Table 1. (1) FB15k-237 [51] is a standard benchmark of encyclopedic facts; small and familiar from rule-mining studies, it hosts the ablation study (Exp-6) and comparisons with published results. (2) ogbl-biokg [27] is a biomedical graph over genes, diseases, drugs and side effects; its relations are dense and normalized, its mined rules are readable, and it hosts the case study. (3) FinReflectKG [4] is a financial graph extracted from the annual 10-K filings of 743 S&P 500 companies, 2014–2024; it provides the motivating scenario of financial risk reasoning at scale. (4) DBpedia [33] closes the ladder; it is the stress test for scalability (Exp-5), on the same graph as GAR discovery [16, 17] for a direct comparison. We also generated synthetic graphs with gMark [5], up to 100M vertices and 1B edges, for the scalability test (Exp-5).

Workloads. Each workload is a set of deduction tasks. A task $(G, \Sigma, \Gamma_0, g, B)$ asks to derive a goal g from initial facts Γ_0 within a budget B. For each dataset, we held out **X** triples as target facts; a task takes one of them as g and the rest of the graph as Γ_0, and we keep only goals that the unrestricted chase derives, so that every task is feasible in principle. The budget B ranges over $\{0.5, 1, 2, 4\} \times B^*$, where B^* is the cost at which the chase derives g: at $B < B^*$ the task tests anytime reasoning, and at $B \geq B^*$ optimal reasoning. We report averages over **X** tasks per dataset.

Baselines. We compared against three families. (1) Rule-evaluation systems: MiniClean [6, 7], which reasons with cleaning rules on GPUs, and HyperBlocker [57], a GPU-accelerated rule engine for entity resolution; both evaluate rules at scale, with neither goal direction nor budgets. (2) Learned graph reasoning: RNNLogic [42] and AnyBURL [37], which learn rules and their weights for link prediction; and DeepPath [55] and MINERVA [12], which learn policies that walk paths toward a query. (3) Unrestricted deduction: the standard chase, which applies all applicable rules to exhaustion. Rule-based baselines are given the same mined rules as our policies. We also evaluated variants of our method: noDAG, which drops the rule signal rel_{fam}; noHDC, which drops the match signal rel_{evd}; and noAdapt, which follows the deterministic policy without budget-aware execution.

Rule libraries and motif construction. For each dataset we mined GARs with the discovery algorithm of [16], at support **X** and confidence **X**, and kept the top-**X** rules by confidence as the rule library; the libraries contain **X**, **X**, **X** and **X** GARs over the four datasets, respectively. As ML predicates we used **X**, trained with its default configuration. Each GAR and its matches induce reasoning motifs.

Evaluation metrics. We report (1) *success rate*, the fraction of tasks that derive g within the budget; (2) *cost*, the total cost of the returned chain, the predicate evaluations it paid; (3) *utility* $U(\Gamma_{\leq B}, g)$ for anytime reasoning; and (4) *runtime* in wall-clock seconds.

Default parameters. Unless stated otherwise: $\alpha = 1$ and $\beta = 1$ in the score f; decay $\gamma = 0.5$ in rel_{fam}; $\nu = 1$ balancing the two signals; temperature $\tau = 1$ for randomized policies; HDC dimension $D = 10^4$; top-$k = 5$ candidates per branch and **X** rollouts per candidate.

Configuration. Experiments were conducted on a server with **X**



Figure 7: Overall performance under varying budgets (columns: success rate, utility, cost and runtime; rows: FB15k-237, ogbl-biokg, FinReflectKG and DBpedia, top to bottom; placeholders to be replaced by plots).

GB RAM and **X** CPUs; GPU baselines ran on **X**. All methods were implemented in **X**. We ran each experiment three times and report the average. Due to space limits, we report results on representative datasets; results on the others are consistent.

Experimental findings.

Exp-1: Overall performance comparison. Figure 7 reports the four metrics over the four datasets as the budget varies. Policy-guided deduction derives **X** % of the goals within budget, **X** points above the strongest baseline, at **X** of its cost; the gain is largest at tight budgets, where execution switches between the two regimes.

Policy types.

State features.

Motif selection strategies.

Soundness and completeness.

Exp-2: Anytime performance and budget analysis

Exp-3: Approximation to optimal chains

Exp-4: Generalization across workloads

Exp-5: Scalability analysis

Exp-6: Ablation study.

Remove fact context, subgoals, proof frontier, budget, and uncertainty one by one to show why $\phi(S, g, \mu)$ matters.

Summary. We find the following. (1) (2) (3) (4)

8 RELATED WORK

Goal-directed rule evaluation. Magic sets rewrite a rule program so that only facts relevant to the query are derived [8]; QSQ and tabling evaluate top-down, memoizing subgoal answers [48, 54]: they answer what is derivable toward g, but price nothing and control no budget. Policy-guided deduction selects which derivations to pay for, and, without top-k truncation, certifies what the selection forgoes.

Heuristic, anytime and budgeted search. A* and its AND/OR variants order a search by an estimate of the remaining cost [26, 36]; bounded-cost search caps the total [45, 50], anytime search returns the best plan found so far [24], Monte Carlo tree search trades exploration against exploitation [11], metareasoning prices computation itself [13, 43, 60], and adaptive submodularity picks tests under a budget [22]. HybNS is branch and bound in this family, its bounds from a static analysis of the rules rather than a per-problem heuristic, and the object priced is a logical derivation.

Learned KG reasoning. Neural reasoners embed paths and subgraphs and rank candidate answers [49, 58, 59]; rule learners mine weighted rules from the graph [37, 38] or learn a rule generator [42]; walk-based agents sample a path to an answer [12, 55]. They score or rank answers, and a wrong one costs a lower rank; we produce certified logical derivations under a budget.

LLM agents on graphs. LLM agents walk a graph in free-form steps, prompting the model to propose and to prune [35, 46, 56], inheriting its fluency and its hallucinations. Section 5.3 instead uses constrained decision models: the choice is among candidate motifs produced by matching, each step verifiable by construction.

Provenance and rule engines. Provenance semirings read a derivation in an algebra of one's choosing [23], ranked enumeration lists answers cheapest first [53], and top-k queries price Datalog

derivations and retrieve the cheapest [14], ranking derivations without a budget, while HybNS certifies within one. The chase computes the full closure of a rule set [2, 9], and modern engines scale it to large knowledge graphs [10, 40]. We keep its semantics and take control of its order: the agent chases only what serves the goal.

9 CONCLUSION

We have proposed a policy-guided deduction framework for agentized graph reasoning. By abstracting policy selection, it unifies optimal and anytime reasoning, supports practical policy classes with adaptive execution, and guarantees correctness, performance and convergence.

REFERENCES

[1] 2026. Full version. *link*.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight Experience Replay. In *NeurIPS*. 5048–5058.

[4] Abhinav Arun, Fabrizio Dimino, Tejas Prakash Agarwal, Bhaskarjit Sarmah, and Stefano Pasquali. 2025. FinReflectKG: Agentic Construction and Evaluation of Financial Knowledge Graphs. *arXiv preprint arXiv:2508.17906* (2025).

[5] Guillaume Bagan, Angela Bonifati, Radu Ciucanu, George H. L. Fletcher, Aurélien Lemay, and Nicky Advokaat. 2017. gMark: Schema-Driven Generation of Graphs and Queries. *IEEE Trans. Knowl. Data Eng.* 29, 4 (2017), 856–869.

[6] Wenchao Bai, Wenfei Fan, Daji Li, Jian Li, Shuhao Liu, Mingliang Ouyang, and Qiang Yuan. 2025. MiniClean: A Single-Machine System for Cleaning Big Graphs. In *SIGMOD (demo)*.

[7] Wenchao Bai, Wenfei Fan, Shuhao Liu, Kehan Pang, Xiaoke Zhu, and Jiahui Jin. 2025. Rule-Based Graph Cleaning with GPUs on a Single Machine. In *SIGMOD*.

[8] Catriel Beeri and Raghu Ramakrishnan. 1991. On the Power of Magic. *J. Logic Programming* 10, 3–4 (1991), 255–299.

[9] C. Beeri and M.Y. Vardi. 1984. A proof procedure for data dependencies. *JACM* 31, 4 (1984), 718–741.

[10] Luigi Bellomarini, Emanuel Sallinger, and Georg Gottlob. 2018. The Vadalog System: Datalog-based Reasoning for Knowledge Graphs. *PVLDB* 11, 9 (2018), 975–987.

[11] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games* 4, 1 (2012), 1–43.

[12] Rajarshi Das, Shehzaad Dhuliawala, Manzil Zaheer, Luke Vilnis, Ishan Durugkar, Akshay Krishnamurthy, Alex Smola, and Andrew McCallum. 2018. Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases Using Reinforcement Learning. In *ICLR*.

[13] Thomas Dean and Mark Boddy. 1988. An Analysis of Time-Dependent Planning. In *AAAI*. 49–54.

[14] Daniel Deutch, Amir Gilad, and Yuval Moskovitch. 2015. Selective Provenance for Datalog Programs Using Top-k Queries. *PVLDB* 8, 12 (2015), 1394–1405.

[15] Wenfei Fan, Lihang Fan, Dandan Lin, and Min Xie. 2025. Explaining GNN-based Recommendations in Logic. *PVLDB* 18, 3 (2025), 715–728.

[16] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Ping Lu, and Chao Tian. 2022. Discovering association rules from big graphs. *PVLDB* 15, 7 (2022), 1479–1492.

[17] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.

[18] Wenfei Fan, Yang Leng, Daji Li, Shuhao Liu, Mingliang Ouyang, Yaoshu Wang, Min Xie, and Qiang Yuan. 2025. DreamCreek: AI for Battery Formation and Grading. In *ICDE*. IEEE, 4544–4547.

[19] Wenfei Fan, Daji Li, Peiyu Liang, Shuhao Liu, Yaoshu Wang, Yiming Wang, Min Xie, and Runjie Zhang. 2024. Graph Association Analyses for Early Drug Discovery. *PVLDB* 17, 12 (2024), 4293–4296.

[20] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[21] Floris Geerts, Giansalvatore Mecca, Paolo Papotti, and Donatello Santoro. 2013. The LLUNATIC Data-Cleaning Framework. *PVLDB* 6, 9 (2013), 625–636.

[22] Daniel Golovin and Andreas Krause. 2011. Adaptive Submodularity: Theory and Applications in Active Learning and Stochastic Optimization. *J. Artif. Intell. Res.* 42 (2011), 427–486.

[23] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *PODS*. 31–40.

[24] Eric A. Hansen and Rong Zhou. 2007. Anytime Heuristic Search. *J. Artif. Intell. Res.* 28 (2007), 267–297.

[25] Stevan Harnad. 1990. The Symbol Grounding Problem. *Physica D: Nonlinear Phenomena* 42, 1–3 (1990), 335–346.

[26] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* 4, 2 (1968), 100–107.

[27] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *NeurIPS*.

[28] Jan Jakubův and Josef Urban. 2017. ENIGMA: Efficient Learning-based Inference Guiding Machine. In *CICM*. 292–302.

[29] Pentti Kanerva. 2009. Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors. *Cognitive Computation* 1, 2 (2009), 139–159.

[30] Zuhair Khayyat, Ihab F. Ilyas, Alekh Jindal, Samuel Madden, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Si Yin. 2015. BigDansing: A System for Big Data Cleansing. In *SIGMOD*. 1215–1230.

[31] Samir Khuller, Anna Moss, and Joseph Naor. 1999. The budgeted maximum coverage problem. *Inf. Process. Lett.* 70, 1 (1999), 39–45.

[32] Donald E. Knuth. 1977. A generalization of Dijkstra's algorithm. *Inf. Process. Lett.* 6, 1 (1977), 1–5.

[33] Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. 2015. DBpedia - A large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web* 6, 2 (2015), 167–195.

[34] Xing Li, Qingcheng Chang, Jinzhong Ning, Changfeng Xu, Shenlong Zhang, Yijia Zhang, Ling Luo, and Hongfei Lin. 2026. Specialized Decision Models vs. General-Purpose LLMs: Benchmarking Jev Across Knowledge, Reasoning, and Multilingual Tasks. *arXiv preprint arXiv:2610.11978* (2026).

[35] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *ICLR*.

[36] Alberto Martelli and Ugo Montanari. 1973. Additive AND/OR Graphs. In *IJCAI*. 1–11.

[37] Christian Meilicke, Melisachew Wudage Chekol, Daniel Ruffinelli, and Heiner Stuckenschmidt. 2019. Anytime Bottom-Up Rule Learning for Knowledge Graph Completion. In *IJCAI*. ijcai.org, 3137–3143.

[38] Christian Meilicke, Manuel Fink, Yanjie Wang, Daniel Ruffinelli, Rainer Gemulla, and Heiner Stuckenschmidt. 2018. Fine-Grained Evaluation of Rule- and Embedding-Based Systems for Knowledge Graph Completion. In *ISWC*. 3–20.

[39] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions - I. *Math. Program.* 14, 1 (1978), 265–294.

[40] Yavor Nenov, Robert Piro, Boris Motik, Ian Horrocks, Zhe Wu, and Jay Banerjee. 2015. RDFox: A Highly-Scalable RDF Store. In *ISWC*. 3–20.

[41] Kehan Pang, Wenfei Fan, Min Xie, and Dandan Lin. 2026. Explaining GNN Negatives Globally and Locally. In *ICDE*.

[42] Meng Qu, Junkun Chen, Louis-Pascal Xhonneux, Yoshua Bengio, and Jian Tang. 2020. RNNLogic: Learning logic rules for reasoning on knowledge graphs. *arXiv preprint arXiv:2010.04029* (2020).

[43] Stuart Russell and Eric Wefald. 1991. Principles of Metareasoning. *Artificial Intelligence* 49, 1–3 (1991), 361–395.

[44] Shenzhen Institute of Computing Sciences. 2022. Fishing Fort. *https://en.sics.ac.cn/col84/index*.

[45] Roni Tzvi Stern, Rami Puzis, and Ariel Felner. 2011. Potential Search: A Bounded-Cost Search Algorithm.. In *ICAPS*.

[46] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.

[47] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 2000. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *NeurIPS*. 1057–1063.

[48] Terrance Swift and David S. Warren. 2012. XSB: Extending Prolog with Tabled Logic Programming. *Theory Pract. Log. Program.* 12, 1–2 (2012), 157–187.

[49] Komal K. Teru, Etienne G. Denis, and William L. Hamilton. 2020. Inductive Relation Prediction by Subgraph Reasoning. In *ICML*. 9448–9457.

[50] Jordan Tyler Thayer, Roni Stern, Ariel Felner, and Wheeler Ruml. 2012. Faster Bounded-Cost Search Using Inadmissible Estimates. In *ICAPS*.

[51] Kristina Toutanova, Danqi Chen, Patrick Pantel, Hoifung Poon, Pallavi Choudhury, and Michael Gamon. 2015. Representing text for joint embedding of text and knowledge bases. In *EMNLP*.

[52] TypeSafe AI. 2026. Introducing System One Models & Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev.

[53] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2021. Beyond Equi-Joins: Ranking, Enumeration and Factorization. *PVLDB* 14, 11 (2021), 2599–2612.

[54] Laurent Vieille. 1986. Recursive Axioms in Deductive Databases: The Query/-Subquery Approach. In *Expert Database Conf.* 253–267.

[55] Wenhao Xiong, Thien Hoang, and William Yang Wang. 2017. DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning. In *EMNLP*.

[56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.

[57] Xiaoke Zhu, Min Xie, Ting Deng, and Qi Zhang. 2025. HyperBlocker: Accelerating Rule-based Blocking in Entity Resolution Using GPUs. *PVLDB* 18, 2 (2025).

[58] Zhaocheng Zhu, Xinyu Yuan, Mikhail Galkin, Sophie Xhonneux, Ming Zhang, Maxime Gazeau, and Jian Tang. 2023. A*Net: A Scalable Path-based Reasoning Approach for Knowledge Graphs. In *NeurIPS*. 59323–59336.

[59] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. 2021. Neural Bellman-Ford Networks: A General Graph Neural Network Framework for Link Prediction. In *NeurIPS*. 29476–29490.

[60] Shlomo Zilberstein. 1996. Using Anytime Algorithms in Intelligent Systems. *AI Magazine* 17, 3 (1996), 73–83.

A PROOF OF THEOREM 1

We first show that both problems admit polynomial-size certificates that can be verified in polynomial time (PTIME). A certificate is a reasoning chain $C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle$. Its length is bounded by the number of distinct motifs, not by B: a failed step or a repeated application can be dropped from a certificate, which still derives g at no larger cost, so a shortest certificate applies each motif at most once, and k is polynomial in $|G|$ and $|\Sigma|$.

Upper bounds. We guess a reasoning chain starting from Γ_0. At each step, we check that: (a) φ_i is applicable under h_i *w.r.t.* the current fact set (predicate evaluation is polynomial in the size of G and Γ), and (b) the derived fact is correctly added. This yields Γ_k (or $\Gamma_{\le B}$) in $O(k \cdot T_{\text{apply}})$ time. Then we verify the following.

- OPSM: We verify that $g \in \Gamma_k$ and $\text{cost}(C) \le B$. Since k is polynomial in $|G|$ and $|\Sigma|$ and each step is polynomial, verification is polynomial.
- BOSP. We additionally compute $U(\Gamma_{\le B}, g)$; U is a weighted coverage of subgoal instances, computable in polynomial time from $\Gamma_{\le B}$. We then check $U(\Gamma_{\le B}, g) \ge \eta$ and $\text{cost}(C) \le B$.

Thus, both problems are in NP.

Lower bounds. We reduce from classical NP-complete problems, building one gadget for both.

OPSM. We reduce from Set Cover [20], under unit cost: every application, successful or not, costs 1. Given a universe $\mathcal{U} = \{e_1, \ldots, e_n\}$, sets $\mathcal{S} = \{S_1, \ldots, S_m\}$ and an integer K, build a graph with a vertex s_i per set and a vertex u_j per element, an edge $\text{member}(s_i, u_j)$ whenever $e_j \in S_i$, and an initial fact $\text{avail}(s_i)$ per set. The rules are: (a) $\text{avail}(x) \to \text{on}(x)$, activating a set; (b) for each $e_j \in S_i$, $\text{member}(x, y) \wedge \text{on}(x) \to \text{cov}(y)$, covering an element of an active set; and (c) $\bigwedge_{j=1}^{n} \text{cov}(y_j) \wedge \bigwedge_{i<j}(y_i \neq y_j) \to g$, with a pairwise precondition $y_i \neq y_j$ per pair, in the idiom of φ_4 of Example 2: matches are homomorphisms, and without them one covered element could serve as every y_j. A chain derives g iff it activates a cover of $\mathcal{U}$, at a cost of $b + n + 1$ for b activated sets, n coverings and the final application; hence a chain of cost at most $K + n + 1$ exists iff a cover of size at most K does.

BOSP. The same gadget reduces from budgeted maximum coverage [31], which asks for at most K sets covering at least η elements: declare each covered element $\text{cov}(u_j)$ a subgoal of weight 1, and set the budget to $K + \eta$ and the threshold to η. A chain of cost at most $K + \eta$ and utility at least η activates b sets and covers $c \ge \eta$ elements at cost $b + c$, so $b \le K$ and the b active sets cover at least η elements; conversely, such sets yield such a chain. The sequential structure is respected: a set must be activated before its elements can be covered.

Both constructions are in PTIME, and solutions correspond to valid reasoning chains. Hence OPSM and BOSP are NP-hard, and NP-complete with the upper bounds. The hardness is in the combined complexity, with the rules part of the input, and rests on shared subderivations being charged once: charged as trees, with shared premises paid anew, the minimum would be computable in polynomial time by a tropical fixpoint [32]. □

B PROOF OF THEOREM 2

The agent may attempt a candidate $\mu = (\varphi, h)$ with untested preconditions, but only a successful application derives: one whose preconditions all hold under h *w.r.t.* the current fact set is a chase step, deriving a fact logically implied by existing facts and Σ, and a failed one adds nothing. Since the chase computes the closure of all such valid applications, every fact derived by the policy must also appear in Γ^*. Thus, policy-guided deduction derives only entailed facts. □

C PROOF OF THEOREM 3

Since $g \in \Gamma^*$, there exists a chase sequence that derives g from Γ_0. We construct a policy that, at each step, selects the next rule application in this sequence. This policy induces a reasoning chain that exactly follows the chase derivation and reaches g. Thus, any entailed goal can be derived by a suitable policy.

Under fairness, at every state, every motif whose preconditions hold is eventually tried: a motif failing on static preconditions never has them hold later and stays retired, and one failing on dynamic preconditions is re-admitted as Γ grows, so no motif of a valid derivation of g is ignored when its turn comes. Since the chase sequence that derives g consists of finitely many such motifs, the policy will eventually apply all of them. Thus, g is derived in finite steps. □

D PROOF OF THEOREM 6

Write U_i for the utility gathered after i steps of the pass, Γ_i for the fact set, and $U^* = U(C^*)$; let O be the at most B' motifs of C^*, each available to the pass by hypothesis. By monotonicity and submodularity,

$$U^* \;\le\; U_i + \textstyle\sum_{\mu \in O} \Delta U(\mu \,|\, \Gamma_i) \;\le\; U_i + B' \cdot \max_\mu \Delta U(\mu \,|\, \Gamma_i),$$

so some available motif adds at least $(U^* - U_i)/B'$, and the chosen one loses at most ϵ against it: $U^* - U_{i+1} \le (1 - 1/B')(U^* - U_i) + \epsilon$. Unfolding the recurrence over B' steps, $U^* - U_{B'} \le (1 - 1/B')^{B'} U^* + B'\epsilon \le U^*/e + B'\epsilon$. □

E PROOF OF THEOREM 5

Call a chain *irredundant* when every step in it derives a fact that is used, as a premise of a later step or as g; an optimal chain is irredundant, since dropping a step that failed or whose fact is never used still derives g at strictly lower cost. Along an irredundant chain, the suffix that begins with a candidate μ projects to a walk from its consequent p_0 to g in $D_{\Sigma,g}$; with edges weighted by w_0, the suffix costs at least $\text{price}(\mu) = w_0(\varphi) + d(p_0)$. A branch is cut only when the cost so far plus this bound exceeds B or reaches $\hat{C}$, so every irredundant chain through a cut branch costs more than B or no less than $\hat{C}$; and a redundant chain is dominated by its irredundant subchain, which the sibling branches explore at strictly smaller cost, each skipped step unpaid. A dynamic failure bars a motif only at the fact set that saw it fail, so no chain is lost to a stale record. With no top-k truncation nothing else is discarded, a chain is recorded only within B, and the search ends only when every branch is cut or completed; hence a chain is recorded whenever one fits B, and no cheaper chain to g within B is missed. □

Algorithm BuildDG
Input: goal g, rule set Σ.
Output: $D_{\Sigma,g} = (N, A)$ and table d.

1. $N \leftarrow \{\text{pred}(g)\}$, $A \leftarrow \emptyset$, $d[\text{pred}(g)] \leftarrow 0$; enqueue $\text{pred}(g)$ into queue Q;
2. **while** Q is not empty **do**
3. dequeue p from Q;
4. **for each** rule $\varphi \in \Sigma$ with consequent p **do**
5. **for each** predicate q in the pattern or preconditions of φ **do**
6. add $q \to p$ to A;
7. **if** $q \notin N$ **then** $N \leftarrow N \cup \{q\}$, $d[q] \leftarrow d[p] + 1$, enqueue q into Q;
8. **return** $D_{\Sigma,g} = (N, A)$ and d.

Figure 8: Building $D_{\Sigma,g}$ and $d(\cdot)$.

Algorithm ScoreF
Input: $D_{\Sigma,g} = (N, A)$ with table d, rule φ of the candidate, decay factor $\gamma \in (0, 1)$.
Output: $\text{rel}_{\text{fam}}(\mu)$.

1. $s \leftarrow 0$;
2. **for each** predicate q of φ (pattern, preconditions and consequent) **do**
3. **if** $q \notin N$ **then return** 0; /* *outside $D_{\Sigma,g}$: filtered out* */
4. $s \leftarrow s + d[q]$;
5. **return** γ^s.

Figure 9: Scoring a candidate from $D_{\Sigma,g}$.

F PROOF OF THEOREM 7

One can verify that the number of distinct ground facts that can be derived is finite (bounded by $|V|$, $|\Sigma|$, and the attribute domains). So is the number of candidate motifs, the matches of Σ in G. Each step either adds a new fact to Γ, no fact added twice, or retires a candidate, recording its failure and never retrying it at the same state; a dynamic failure is re-admitted only when Γ has grown, so each motif is tried at most once per distinct fact set, at most $|\Gamma^*| + 1$ sets in all. Both pools are finite, hence the reasoning process performs finitely many steps, at most the number of motifs times $|\Gamma^*| + 1$, and must terminate. □

G BOUNDED COST AND CONVERGENCE

Theorem 8: *Let $m = \max_S |\mathcal{A}(S)|$ be the largest candidate set, T_{score} the cost of scoring one candidate, and T_{eval} the cost of one application. Then the total runtime of policy-guided deduction over B steps is $O\big(B \cdot (m \cdot T_{score} + T_{eval})\big)$.* □

Proof sketch: Each step scores the at most m candidates in $O(m \cdot T_{\text{score}})$ and evaluates the selected one in T_{eval}. Over B steps, this yields $O(B \cdot (m \cdot T_{\text{score}} + T_{\text{eval}}))$. Matching maintenance is control overhead: it adds to the wall-clock beyond this bound, never to the budget. □

Theorem 9: *If execution continues without a budget constraint and the policy eventually explores all applicable motifs (i.e., is fair), then the resulting fact set converges to the chase closure Γ^*.* □

Proof sketch: By fairness, every applicable motif is eventually applied, hence all facts in Γ^* are derived. Since derivations are sound, no extra facts are produced, so the limit equals Γ^*. Termination implies convergence to this fixed point. □

H ALGORITHMS FOR THE DEPENDENCY GRAPH

Algorithm BuildDG (Figure 8) constructs the dependency graph $D_{\Sigma,g}$ and the distance table $d(\cdot)$, by a backward BFS from the goal over the rules; when rule edges are weighted, the BFS becomes a Dijkstra computation, relaxing $d[q] \leftarrow \min(d[q], d[p] + w_0(\varphi))$ at line 7, where $w_0(\varphi)$ is the certified lower bound on the cost of one application of φ, at least 1. It visits each rule once per predicate of the rule, *i.e.*, in $O(|\Sigma|)$ time. The table also bounds completion: any chain through a candidate with consequent p_0 completes at cost at least $d(p_0)$, since each application on the way costs at least its edge weight and the path is shortest. The completion price of a candidate is $\text{price}(\mu) = w_0(\varphi) + d(p_0)$: its own application, certified below by w_0, plus the cheapest completion thereafter. Algorithm ScoreF (Figure 9) scores a candidate motif $\mu = (\varphi, h)$ from the table. The match h plays no role here: rel_{fam} is a property of the rule, and the match enters only in rel_{evd}.

The learned policies read the graph through a vector $e_D = \bigoplus_k \rho^k(b_k)$, where b_k bundles the vectors of the predicates k rule applications from g; shifted onto their own channels, with each predicate entering the channel of its nearest depth, the shifts interfere only through their quasi-orthogonality, priced in Appendix I, and a predicate that cannot reach g never enters e_D. The vector is settled in the same backward pass as $d(\cdot)$, once per goal; for Example 6, $e_D = v_{\text{HighRisk}} \oplus \rho(b_1)$, with b_1 bundling owns, transfers, amount $> \theta$, M_{anom} and uses. A candidate bundles the predicates of its rule, premises and consequent alike, each once, into $v_\varphi = \bigoplus_{q \in \varphi} v_q$ and probes e_D per depth: the cosine $s_k = \cos(\rho^k(v_\varphi), e_D)$ reads how many of them sit k applications from g, up to the error of Appendix I, the shift ρ^k singling out the depth-k part of e_D. The probes give a profile $(s_0, s_1, \dots)$ per candidate, *e.g.*, proportional to $(s_0, s_1) = (1, 4), (1, 3), (1, 1)$ for candidates of $\varphi_1, \varphi_2, \varphi_3$ of Example 6, and $(0, 1)$ for φ_4, whose other premises sit on no channel. The profile is fed whole to f_θ, a fixed-width encoding that generalizes across predicates and goals, where the handcrafted rel_{fam} reads the same distances exactly, from the table (Figure 9), *e.g.*, the $\gamma^4, \gamma^3, \gamma$ of Example 6.

I ERROR OF COSINE READINGS

We justify the counts read from cosines in Section 4: the reading error is worst-case, data-free, and shrinks as the dimension grows, apart from an explicit collision bias. Each symbol carries a vector of D independent coordinates, uniform in $\{-1/\sqrt{D}, +1/\sqrt{D}\}$, hence of unit norm, and distinct symbols are independent; the vectors are generated on the fly from identifiers by hashing, and nothing is materialized per vertex; fact vectors are normalized to unit length, and each predicate enters the dependency encoding at its nearest depth only.

Lemma 10: *Let the probe u bundle m premise vectors and E bundle n fact vectors, all of unit length, let c count the premise facts occurring in E, let s count spurious symbol collisions — pairs of distinct facts, one in each bundle, that share a predicate or a vertex in the same role position without sharing the fact — and write $\tilde{s} = \sum 1/(1+a)$ over the same pairs, a the arity of the colliding facts. For $m + n \le \sqrt{D}$, with probability at least $1 - \delta$ over the random vectors, $|\cos(u, E) - (c + \tilde{s})/\sqrt{mn}| \le \sqrt{2\ln(2/\delta)/D}\,(1 + o(1))$.* □

Proof sketch. Write $\cos(u, E) = \langle u, E\rangle / (\|u\|\,\|E\|)$. The inner product splits into a signal c from the shared facts, a collision term $\tilde{s}$, and a noise of independent terms of $\pm 1/D$, at most mnD of them: a pair of distinct arity-a facts sharing a role-position symbol contributes exactly $1/(1+a)$, deterministically, the shared symbol carrying that weight in each unit-length fact vector. By Hoeffding the noise exceeds $\sqrt{2mn\ln(2/\delta)/D}$ with probability at most δ. The norms concentrate as $\|u\| = \sqrt{m}\,(1 + o(1))$ and $\|E\| = \sqrt{n}\,(1 + o(1))$ for $m + n \le \sqrt{D}$. Dividing yields the bound, which is free of m and n. When no predicate-role or vertex-role pair repeats across the bundles, $s = 0$ and the reading recovers c; realistic chains may repeat predicates, and the reading then over-counts by $\tilde{s}/\sqrt{mn}$, at most $s/\sqrt{mn}$, a bias, not noise. □

The bound prices both readings of Section 4. For the evidence score, m is the few premises of a candidate and n the facts of the chain: adjacent counts differ by $1/\sqrt{mn}$, against a noise of $\sqrt{2\ln(2/\delta)/D}$, *e.g.*, a third of that step at $m = 1$, $n = 100$, $D = 10^4$ and $\delta = 0.01$; an absent premise has $c = 0$, hence $|\cos| \le 0.033$ when no role repeats; the bias stays small when repeated roles within a bundle are few, and when a chain repeats one predicate in the same role, *e.g.*, long HighRisk propagations, s grows like mn and the reading over-counts systematically, to be quantified in the experiments. The depth probes s_k read the same way: the dependency encoding bundles N predicates on disjoint channels, and the cosine reads $c_k/\sqrt{mN}$ against the same noise, reliable while $m + N$ stays below $\sqrt{D}$, so D grows with $(m+N)^2$ for larger libraries. The readings are worst-case over the random vectors and need no data; a learned embedding reads nothing of what its scores count.